Python Functions , Modular Code And Lambda Functions

Suman 3 Feb, 2026 At 10:00 PM Notes Recommended Resource

Associated Lectures (1)

Description Discussions

Session 2.3: Functions & Modular Code

In-Class Resources

Lecture Notes

Program: AIM101 — Artificial Intelligence & Machine Learning Foundation **Institution:** Vishlesan i-Hub IIT Patna × Masai School

Mental Map

```
SESSION 2.2: FUNCTIONS & MODULAR CODE
============================================================

PART 1: OPENING (8 min)
  Revision (5 min) → Code Repetition Hook (3 min)

PART 2: CORE CONTENT – FIRST HALF (40 min)
  Section A: Defining Functions & Parameters (12 min)
       └── def keyword, function anatomy, parameters vs arguments

  Section B: Return Values (10 min)
       └── return statement, return vs print, multiple returns

  Section C: Default & Keyword Arguments (10 min)
       └── Default values, keyword arguments, mixing approaches

  Section D: Variable Scope (8 min)
       └── Local vs Global, LEGB rule, avoiding global modification

  === IN-CLASS QUIZ (7.5 min) ===

PART 3: SECOND HALF (32 min)
  Section E: Docstrings & Best Practices (10 min)
       └── Documentation, naming conventions, professional habits
```

```
  Section F: Lambda Functions (10 min)
       └── Anonymous functions, sorted/map/filter usage

  Section G: Importing Modules (12 min)
       └── import patterns, math, random, datetime

PART 4: MINI-PROJECT (14 min)
  Calculator with Functions
       └── Modular design, refactoring, code organization

WRAP-UP (3 min)
  Key takeaways → Session 2.3 preview

============================================================
Total: 105 minutes
```

The Big Picture

Functions are the building blocks of organized, maintainable code. Without functions, you would copy-paste the same logic every time you need it, leading to code that is longer, harder to maintain, and prone to bugs. If you needed to change a formula, you would have to find and update it in dozens of places. Miss one location, and you have introduced a bug.

This session teaches you to write code once and reuse it anywhere. A function is like a recipe: you write the steps once, give it a name, and then follow it whenever you need that result. Functions make your code shorter, cleaner, and much easier to debug.

Beyond basic function definition, you will learn professional practices that separate beginner code from production-ready code: documenting your functions with docstrings, understanding variable scope to avoid mysterious bugs, using lambda functions for quick operations, and leveraging Python's rich library of modules. By the end of this session, you will be able to refactor repetitive code into elegant, reusable functions and build modular programs that are easy to understand and maintain.

Main Content

Section A: Defining Functions & Parameters

A function is a named block of reusable code. The `def` keyword tells Python you are about to define a function. The function name follows Python naming conventions (snake_case), and parameters go inside parentheses.

Function Anatomy

```python
def greet(name):              # def + function_name + (parameters):
    message = f"Hello, {name}!"   # Function body (indented)
    print(message)                # What the function does
```

```
greet("Raj")                # Function CALL with argument
```

Key parts:

- `def` - Keyword that starts a function definition
- `greet` - Function name (you choose this)
- `name` - Parameter (placeholder for a value)
- `:` - Required colon ending the function header
- Indented code - The function body
- `"Raj"` - Argument (actual value passed when calling)

Parameters vs Arguments

This distinction is crucial:

- **Parameter:** Variable in the function DEFINITION (the placeholder)
- **Argument:** Actual value in the function CALL (what fills the placeholder)

Think of it like parking: a parameter is the parking spot, an argument is the car that fills it.

```
def greet(name):        # 'name' is a PARAMETER
    print(f"Hello, {name}!")

greet("Raj")            # "Raj" is an ARGUMENT
greet("Priya")          # "Priya" is an ARGUMENT
```

Multiple Parameters

Functions can accept multiple parameters, separated by commas. When calling the function, arguments are matched to parameters by position.

```
def introduce(name, age, city):
    print(f"My name is {name}.")
    print(f"I am {age} years old.")
    print(f"I live in {city}.")

# Correct order
introduce("Raj", 22, "Delhi")

# Wrong order causes wrong output!
introduce("Delhi", "Raj", 22)
# Output: My name is Delhi. I am Raj years old. I live in 22.
```

Key insight: Order matters with positional arguments. The first argument goes to the first parameter, second to second, and so on.

Section B: Return Values

The `return` statement sends a value back to the code that called the function. This is different from `print()`, which only displays output.

Basic Return

```
def add(a, b):
    result = a + b
    return result

# Store the returned value
sum1 = add(5, 3)      # sum1 = 8
sum2 = add(10, 20)    # sum2 = 30

# Use returned values in further calculations
total = sum1 + sum2   # total = 38
```

Return vs Print - Critical Difference

This is a common source of confusion for beginners:

```
def multiply_print(a, b):
    print(a * b)      # Displays 20, returns None

def multiply_return(a, b):
    return a * b      # Returns 20

# Using print version
result1 = multiply_print(4, 5)   # Shows: 20
print(f"result1 = {result1}")    # Shows: result1 = None

# Using return version
result2 = multiply_return(4, 5)
print(f"result2 = {result2}")    # Shows: result2 = 20
print(f"Double: {result2 * 2}")  # Shows: Double: 40
```

Rule: Use `print()` for displaying to humans. Use `return` when you need the value for further computation.

Returning Multiple Values

Python allows returning multiple values, which are returned as a tuple and can be unpacked:

```
def get_min_max(numbers):
    minimum = min(numbers)
    maximum = max(numbers)
    return minimum, maximum  # Returns a tuple

data = [45, 22, 89, 12, 67, 34]
low, high = get_min_max(data)   # Unpacking

print(f"Minimum: {low}")   # 12
print(f"Maximum: {high}")  # 89
```

Early Return Pattern

Use `return` to exit a function early, often for error handling:

```
def divide(a, b):
    if b == 0:
        print("Error: Cannot divide by zero!")
        return None  # Early exit
    return a / b

result = divide(10, 0)  # Prints error, returns None
```

Section C: Default & Keyword Arguments

Default Parameter Values

Default values provide fallbacks when arguments are not supplied:

```
def greet(name, greeting="Hello"):
    print(f"{greeting}, {name}!")

greet("Raj")              # Uses default: Hello, Raj!
greet("Raj", "Hi")        # Overrides: Hi, Raj!
greet("Raj", "Welcome")   # Overrides: Welcome, Raj!
```

Important rule: Parameters with defaults MUST come AFTER parameters without defaults.

```
# VALID
def func(required, optional="default"):
    pass

# INVALID - causes SyntaxError
def func(optional="default", required):
    pass
```

Keyword Arguments

Keyword arguments let you specify which parameter receives which value by name:

```
def create_profile(name, age, city="Unknown", occupation="Student"):
    print(f"{name}, {age}, {city}, {occupation}")

# Positional arguments (order matters)
create_profile("Raj", 22, "Delhi", "Engineer")

# Keyword arguments (order does NOT matter)
create_profile(occupation="Doctor", name="Priya", city="Mumbai", age=28)
```

```
# Skip middle optional parameter
create_profile("Amit", 25, occupation="Designer")  # city stays "Unknown"
```

Best practice: Use keyword arguments for clarity, especially when functions have many parameters.

Section D: Variable Scope

Local vs Global Variables

- **Local variables:** Defined inside a function, exist only during function execution
- **Global variables:** Defined outside all functions, accessible throughout the program

```
message = "I am global"    # Global variable

def show_scope():
    local_msg = "I am local"  # Local variable
    print(f"Inside function:")
    print(f"  Global: {message}")     # Can access global
    print(f"  Local: {local_msg}")    # Can access local

show_scope()

print(f"\nOutside function:")
print(f"  Global: {message}")  # Works
# print(f"  Local: {local_msg}")  # NameError! local_msg does not exist here
```

Variable Shadowing

When a local variable has the same name as a global variable, they are different variables:

```
x = 100  # Global x

def test_shadow():
    x = 50  # Local x (different variable!)
    print(f"Inside function: x = {x}")  # 50

print(f"Before function: x = {x}")  # 100
test_shadow()                        # 50
print(f"After function: x = {x}")   # 100 (unchanged!)
```

The Global Keyword (Use Sparingly)

To modify a global variable inside a function, use the `global` keyword:

```
counter = 0

def increment_bad():
    global counter  # Declare intention to modify global
    counter += 1
```

```
increment_bad()
increment_bad()
print(counter)  # 2
```

Professional advice: Avoid modifying globals. Instead, pass values as arguments and return results:

```
def increment_good(value):
    return value + 1

counter = 0
counter = increment_good(counter)  # counter = 1
counter = increment_good(counter)  # counter = 2
```

This approach makes code easier to understand, test, and debug.

Section E: Docstrings & Best Practices

Writing Docstrings

Docstrings are triple-quoted strings immediately after the `def` line that document what a function does:

```
def calculate_bmi(weight_kg, height_m):
    """
    Calculate Body Mass Index (BMI).

    Args:
        weight_kg: Weight in kilograms
        height_m: Height in meters

    Returns:
        BMI value (weight / height^2)
    """
    bmi = weight_kg / (height_m ** 2)
    return round(bmi, 1)

# Access docstring
print(calculate_bmi.__doc__)
help(calculate_bmi)
```

Function Best Practices

One function = one task (Single Responsibility Principle)

Use descriptive names with verb_noun pattern:

- Good: `calculate_area()`, `get_user_input()`, `validate_email()`
- Bad: `func()`, `do_stuff()`, `x()`

Keep functions short (ideally under 20 lines)

Always write docstrings

Avoid modifying global variables

Section F: Lambda Functions

Lambda functions are small, anonymous functions defined in a single line.

Basic Syntax

```
# Regular function
def square_regular(x):
    return x ** 2

# Lambda equivalent
square_lambda = lambda x: x ** 2

# Both work identically
print(square_regular(5))  # 25
print(square_lambda(5))   # 25
```

Lambda with sorted()

Lambdas are commonly used for custom sorting:

```
students = [
    ("Raj", 85),
    ("Priya", 92),
    ("Amit", 78),
    ("Zara", 95)
]

# Sort by score (second element)
by_score = sorted(students, key=lambda x: x[1])
# Result: [('Amit', 78), ('Raj', 85), ('Priya', 92), ('Zara', 95)]

# Sort by score descending
by_score_desc = sorted(students, key=lambda x: x[1], reverse=True)
```

Lambda with map() and filter()

```
numbers = [1, 2, 3, 4, 5]

# map() applies function to every item
squared = list(map(lambda x: x ** 2, numbers))
# Result: [1, 4, 9, 16, 25]

# filter() keeps items that pass a test
evens = list(filter(lambda x: x % 2 == 0, numbers))
# Result: [2, 4]
```

Caution: Do not overuse lambdas. If your lambda becomes complex, write a regular function instead. Readability matters more than cleverness.

Section G: Importing Modules

Modules are pre-written code packages that extend Python's capabilities.

Import Methods

```
# Method 1: Import entire module
import math
print(math.sqrt(16))     # Must use prefix

# Method 2: Import specific functions
from math import sqrt, pi
print(sqrt(16))          # No prefix needed
print(pi)                # 3.14159...

# Method 3: Import with alias
import math as m
print(m.sqrt(16))        # Shorter prefix
```

The math Module

Essential mathematical functions:

```
import math

math.sqrt(16)      # 4.0 - Square root
math.ceil(4.2)     # 5 - Round up
math.floor(4.8)    # 4 - Round down
math.pi            # 3.14159... - Pi constant
math.pow(2, 3)     # 8.0 - Power function
```

The random Module

For random values (games, testing, simulations):

```
import random

random.random()           # Float between 0 and 1
random.randint(1, 10)     # Integer between 1 and 10 (inclusive)
random.choice(my_list)    # Random item from list
random.shuffle(my_list)   # Shuffle list in place
random.sample(my_list, 3) # 3 random items (no repeats)
```

The datetime Module

For working with dates and times:

```
from datetime import datetime, date, timedelta

now = datetime.now()          # Current date and time
today = date.today()          # Current date only
```

```
formatted = now.strftime("%d/%m/%Y %H:%M")  # Format as string

tomorrow = today + timedelta(days=1)    # Date arithmetic
next_week = today + timedelta(weeks=1)
```

Key Frameworks

Function Anatomy Template

```
def function_name(parameters):
    """Docstring explaining what the function does."""
    # Function body
    return value
```

Data Structure Selection Guide (Review from 2.1)

Need	Use
Ordered, mutable collection	List `[]`
Ordered, immutable collection	Tuple `()`
Key-value lookup	Dictionary `{}`
Unique items, fast membership	Set `{}`

When to Use Lambda vs Regular Function

Scenario	Use
Simple, one-line operation	Lambda
Need docstring or multiple statements	Regular function
Custom sorting key	Lambda
Complex logic or conditionals	Regular function
Reusing the function multiple times with a name	Regular function

Import Statement Patterns

Pattern	Usage
`import module`	When using many functions from module
`from module import func`	When using only specific functions
`import module as alias`	When module name is long

Pattern	Usage
`from module import *`	Avoid this (pollutes namespace)

Common Errors Quick Reference

Error	Cause	Fix
`TypeError: function() takes X positional arguments but Y were given`	Wrong number of arguments	Check function definition and call
`TypeError: function() missing required positional argument`	Forgot to pass required argument	Provide all required arguments
`NameError: name 'x' is not defined`	Using local variable outside function	Return the value instead
`UnboundLocalError: local variable referenced before assignment`	Trying to modify global without `global` keyword	Use `global` keyword or pass as argument
`SyntaxError: non-default argument follows default argument`	Required parameter after optional	Put required parameters first
`TypeError: 'NoneType' object is not...`	Function returns None (no return statement)	Add return statement
`ModuleNotFoundError`	Module not installed	Install with pip or check spelling

Key Takeaways

Functions eliminate code repetition. Write logic once, use it anywhere with different inputs.

`def` creates a function. Follow with function name, parameters in parentheses, and colon.

Parameters are placeholders; arguments are actual values. Parameters in definition, arguments in call.

`return` gives values back to code; `print()` displays for humans. If you need to use a result later, return it.

Multiple return values come as a tuple. Unpack them into separate variables.

Default parameters must come after required parameters. This is a syntax rule.

Keyword arguments improve readability. Especially useful with many parameters.

Local variables exist only during function execution. They are created when the function runs and destroyed when it ends.

Avoid modifying global variables. Pass values in as arguments and return results instead.

Docstrings document your functions. Use triple quotes immediately after `def`. Access with `help()` or `__doc__`.

Lambda functions are for simple, one-line operations. Do not overuse them; readability matters.

Modules extend Python's power. Use `import` to access math, random, datetime, and hundreds of other modules.

Next Session Preview

Session 2.3: [PROFESSOR] Git & Software Engineering Principles

- Why version control matters (disaster recovery stories)
- Git mental model: commits as snapshots, branches as timelines
- Collaboration workflows: fork, pull request, merge
- Code review culture and philosophy
- Documentation importance in professional settings
- Industry practices for ML teams
- Preview: Hands-on Git practice in Week 3

This is a theoretical session (no code). We will understand the concepts behind version control and professional collaboration before doing hands-on Git work in the following week.

Vishlesan i-Hub IIT Patna × Masai School